

Disciplina: Sistemas Distribuídos

Análise Comparativa de Tecnologias de Comunicação: Sockets TCP, RMI e RPC em um Sistema Hospitalar

Gabriel Davi Mota Baia
Julia Veloso Dias

Código-fonte e Implementação:

<https://github.com/GabrielDavi7/Hospital-scheduler-distributed>

1 A Aplicação Escolhida

A aplicação escolhida para este estudo de caso é um **Sistema de Agendamento Hospitalar Distribuído**. O software gerencia a entidade **Consulta**, que encapsula atributos como dados do paciente (Nome, CPF), informações médicas (CRM, Médico) e detalhes do agendamento (Data, Hora). O sistema simula um ambiente real de clínica médica, suportando operações de inserção em massa (para simular picos de tráfego na recepção) e operações de leitura com aplicação de filtros específicos por CPF ou CRM. A natureza transacional e de consulta desta aplicação a torna o cenário ideal para testar os limites de tráfego e latência de rede.

2 Arquitetura de Cada Versão

2.1 Versão 1: Sockets TCP

A versão com sockets utiliza diretamente as classes **ServerSocket** e **Socket** do Java para estabelecer a comunicação TCP entre cliente e servidor. O **ServidorSocket** aguarda conexões na porta 5000, recebe uma solicitação, processa a operação e devolve uma resposta. Nesta implementação, o atendimento é sequencial: o servidor encerra a conexão após responder e então aguarda o próximo cliente.

A aplicação define seu próprio protocolo de mensagens em texto. As operações são identificadas por comandos. No agendamento, os dados são enviados em uma linha com campos separados por ponto e vírgula. O servidor utiliza o ponto e vírgula que separa os campos para interpretar a mensagem e construir o objeto de consulta.

Essa abordagem exige que a própria aplicação controle o formato das mensagens, a interpretação dos campos e o ciclo de abertura e fechamento das conexões. Por exemplo, o cliente abre uma conexão para listar médicos e outra para agendar a consulta, ambas usam o mesmo endereço e a mesma porta do servidor, mas são conexões TCP distintas.

Embora o TCP ofereça transporte confiável e ordenado, a identificação das operações e a organização dos dados são responsabilidade do código da aplicação.

2.2 Versão 2: Java RMI — Remote Method Invocation

A versão RMI utiliza o modelo de objetos distribuídos do Java, permitindo que o cliente execute métodos que estão implementados no servidor como se estivesse chamando métodos de um objeto local. Para isso, a interface `InterfaceRMI` estende `Remote` e define quais operações podem ser acessadas remotamente, como agendar consultas, listar médicos e realizar buscas por CPF ou CRM. A classe `ServidorRMI` implementa essa interface e estende `UnicastRemoteObject`, tornando sua instância disponível para receber chamadas vindas de outros processos pela rede.

Quando o servidor é iniciado, ele cria um RMI Registry na porta 1099 e registra o serviço com um nome conhecido, como `HospitalService`. O Registry pode ser entendido como um catálogo de serviços. O cliente consulta esse catálogo utilizando o método `lookup()` e recebe uma referência para o objeto remoto. A partir dessa referência, ele pode chamar métodos como `agendarConsulta()` ou `buscarPorCpf()` sem precisar criar e interpretar mensagens de texto manualmente. A própria infraestrutura RMI é responsável por enviar os parâmetros ao servidor, executar o método solicitado e devolver o resultado ao cliente.

Para que objetos possam ser enviados entre cliente e servidor, eles precisam ser serializáveis. No projeto, a classe `Consulta` implementa `Serializable`, permitindo que seus dados sejam transformados em uma representação que possa ser transmitida pela rede. Dessa forma, o cliente pode enviar diretamente um objeto `Consulta` no momento do agendamento, e o servidor pode retornar listas de consultas nas operações de busca. Isso evita a necessidade de separar os dados manualmente com delimitadores, como ocorre na versão com sockets. Os objetos recebidos pelo cliente representam cópias dos dados transmitidos, enquanto a agenda principal continua armazenada na memória do servidor.

A porta 1099 é utilizada especificamente pelo RMI Registry, ou seja, ela serve para que o cliente localize o serviço registrado. Após o método `lookup()`, as chamadas são direcionadas ao próprio objeto remoto. Como o construtor de `ServidorRMI` chama `super()` sem informar uma porta específica, o RMI escolhe automaticamente uma porta disponível para receber essas chamadas. O cliente não precisa conhecer essa porta, pois as informações necessárias já estão presentes na referência remota obtida pelo Registry.

2.3 Versão 3: RPC — Apache XML-RPC

A versão RPC utiliza a biblioteca Apache XML-RPC para permitir que o cliente execute procedimentos que estão implementados no servidor. A comunicação ocorre por HTTP, enquanto o conteúdo das requisições e respostas é representado em XML. No projeto, a classe `ServidorRPC` inicia um `WebServer` na porta 8080 e registra o serviço com o nome `HospitalService`. Esse nome é usado pelo cliente para indicar qual procedimento remoto deseja executar, como `HospitalService.agendarConsulta`.

As operações disponíveis são definidas na interface `InterfaceRPC`. No lado do

cliente, a classe `ConexaoRPC` configura o endereço `http://127.0.0.1:8080/RPC2` e utiliza um método para realizar as chamadas remotas. O cliente informa o nome do procedimento e os parâmetros necessários, e a biblioteca Apache XML-RPC é responsável por transformar esses dados em uma mensagem XML, enviá-la ao servidor por HTTP e interpretar a resposta recebida. Assim, o código da aplicação não precisa abrir sockets manualmente nem montar diretamente o conteúdo XML da requisição.

Como o protocolo XML-RPC trabalha com um conjunto específico de tipos de dados, os objetos da aplicação precisam ser convertidos para estruturas compatíveis antes da transmissão. No projeto, uma consulta é representada por um `Map<String, String>`, no qual cada informação da consulta é associada a uma chave. Quando uma busca retorna várias consultas, o resultado pode ser representado por um array contendo esses mapas. A biblioteca converte essas estruturas Java para os tipos correspondentes no XML-RPC durante a comunicação.

Essa forma de transmissão é diferente da utilizada pelo RMI. No RMI, objetos Java podem ser enviados por meio da serialização. No XML-RPC, os dados são convertidos para estruturas padronizadas pelo protocolo. Por isso, o serviço não depende diretamente da serialização de classes Java e, em princípio, poderia ser acessado por clientes desenvolvidos em outras linguagens que também ofereçam suporte ao protocolo XML-RPC.

No servidor, o projeto reutiliza a mesma instância de `ServidorRPC` em todas as chamadas. Dessa forma, a agenda permanece armazenada em memória e pode ser acessada pelas diferentes requisições recebidas. Como mais de uma chamada pode ocorrer ao mesmo tempo, os métodos que manipulam a agenda são declarados como `synchronized`. Isso impede que duas chamadas modifiquem simultaneamente a mesma estrutura de dados, ajudando a evitar inconsistências na lista de consultas e na geração dos identificadores.

Por fim, cada chamada RPC envolve algumas etapas adicionais: os dados precisam ser convertidos para XML, enviados por HTTP, processados pelo servidor e depois convertidos novamente na resposta. Essas etapas fazem parte do custo de comunicação da aplicação e, consequentemente, influenciam o tempo de resposta observado nos testes de desempenho.

3 Dificuldades Encontradas na Implementação

Durante o desenvolvimento das três versões, a equipe deparou-se com desafios técnicos específicos para cada tecnologia:

- **Sockets:** A maior dificuldade foi gerenciar manualmente o formato da mensagem. A equipe precisou concatenar os dados usando ponto e vírgula e reconstruir os objetos usando a função `.split(";")`. Além disso, ao realizar o teste de estresse, a criação sequencial de `Sockets` causou a exaustão rápida das portas efêmeras do sistema operacional, resultando em falhas de `Address already in use`.
- **RMI:** A principal barreira inicial foi a necessidade de manter uma interface estrita (`Remote`) idêntica no cliente e no servidor. Adicionalmente, forçar a

classe `Consulta` a implementar `Serializable` revelou-se um desafio arquitetural, pois o custo da serialização gerou uma lentidão inesperada durante as leituras massivas do sistema.

- **RPC:** Como o projeto não utilizava gerenciadores de dependências automáticos (ex: Maven), a principal dificuldade foi configurar o ambiente. A equipe precisou desenvolver um *script* (`compile-rpc.ps1`) para automatizar o *download* das bibliotecas `xmlrpc-common`, `client` e `server`. No código, o desafio foi desconstruir o objeto Java nativo e adaptar a lógica para trabalhar exclusivamente com o mapeamento genérico `Map<String, String>`.

4 Vantagens e Desvantagens Percebidas

4.1 Sockets TCP

- **Vantagens:** Zero dependência de bibliotecas externas. Entrega a latência mais baixa em requisições únicas e na leitura de grandes volumes de texto puro.
- **Desvantagens:** Nível de abstração nulo. O código fica sujo com a manipulação manual de *strings*. Falta de gestão nativa de fila de conexões leva a problemas críticos de estabilidade sob carga (*Port Exhaustion*).

4.2 Java RMI

- **Vantagens:** Máxima facilidade de uso para quem programa em Java. Alto nível de abstração. O código fica estruturado e seguro (*Type Safety*). Gerencia as portas TCP internamente, evitando quedas de conexão sob estresse.
- **Desvantagens:** Fortemente acoplado à linguagem Java. O custo computacional da serialização/desserialização de objetos causa um severo estrangulamento de performance ao trafegar longas listas de dados na leitura.

4.3 Apache XML-RPC

- **Vantagens:** Possui interoperabilidade, permitindo que o servidor Java seja facilmente consumido por clientes em Python, C# ou JavaScript. O uso de tipos primitivos garantiu maior agilidade na leitura filtrada em comparação ao RMI.
- **Desvantagens:** Apresentou o maior tempo absoluto durante operações de inserção em massa. A construção de arquivos XML sobre o protocolo HTTP gera um *overhead* de rede e processamento muito mais pesado do que as chamadas nativas (Sockets e RMI).

5 Resultados de Desempenho

Para complementar a análise das três arquiteturas, foram realizados testes de desempenho utilizando as implementações com Sockets TCP, Java RMI e Apache XML-RPC. Os intervalos de execução foram medidos utilizando `System.nanoTime()`, permitindo observar o comportamento das tecnologias durante o povoamento da agenda e as operações de busca.

O teste de povoamento consistiu na realização de 10.000 agendamentos sequenciais. Nesse cenário, a versão RMI apresentou o menor tempo total, com 1586 ms. A versão com Sockets realizou a mesma operação em 3316 ms, enquanto a implementação RPC apresentou o maior tempo, com 5101 ms.

Arquitetura	Povoar agenda (10k)	Buscar CPF	Buscar CRM
Socket	3316 ms	37 ms	50 ms
RMI	1586 ms	23,237 ms	28,481 ms
RPC	5101 ms	139 ms	152,8 ms

Tabela 1: Comparativo de desempenho entre Sockets, RMI e RPC.

Os resultados do povoamento mostram comportamentos diferentes entre as arquiteturas. Na implementação avaliada, o RMI apresentou o menor tempo para realizar os 10.000 agendamentos sequenciais, seguido pelos Sockets. O RPC apresentou o maior tempo nessa operação.

Uma possível razão para o maior custo observado no RPC está relacionada às etapas adicionais necessárias para realizar cada chamada. Os parâmetros precisam ser convertidos para estruturas compatíveis com XML-RPC, representados em XML e enviados utilizando HTTP. No servidor, esses dados precisam ser interpretados antes da execução do procedimento, e o mesmo processo ocorre no sentido inverso para a resposta.

Na versão com Sockets, a comunicação é realizada de maneira mais direta utilizando TCP. Entretanto, a própria aplicação precisa controlar a criação das conexões, o formato das mensagens e a interpretação dos dados recebidos. Como cada operação abre uma conexão, envia a requisição, recebe a resposta e encerra a comunicação, esse ciclo também faz parte do tempo observado durante a execução dos 10.000 agendamentos.

O RMI, por sua vez, apresentou 1586 ms para o povoamento da agenda. Nesse modelo, o cliente trabalha diretamente com a referência do objeto remoto e pode enviar objetos Java como parâmetros das chamadas. A infraestrutura RMI realiza automaticamente etapas como serialização, transmissão dos parâmetros, execução do método remoto e retorno do resultado.

5.1 Resultados das buscas

Durante a primeira execução dos testes da versão RMI, foi identificado um problema na metodologia de medição. Os valores inicialmente registrados para as buscas incluíam não apenas a chamada remota, mas também a digitação dos dados pelo usuário e a impressão de todos os resultados no terminal.

Por esse motivo, os valores anteriormente obtidos de 1474 ms para a busca por CPF e 2043 ms para a busca por CRM não representavam exclusivamente o desempenho da chamada RMI.

Após a correção, a medição da versão RMI foi dividida em três partes:

- tempo de processamento da busca no servidor;
- tempo total da chamada remota observado pelo cliente, incluindo envio da solicitação e recebimento da resposta;
- tempo utilizado para formatar e imprimir os resultados no terminal.

Com a nova medição, a busca por CPF retornou 2012 consultas e apresentou tempo total de chamada RMI de 23,237 ms. A busca por CRM retornou 3346 consultas e apresentou tempo de chamada de 28,481 ms.

Comparando esses valores com as demais implementações, a versão Socket apresentou 37 ms na busca por CPF e 50 ms na busca por CRM. Já a implementação RPC registrou 139 ms e 152,8 ms, respectivamente.

Assim, considerando os resultados obtidos nesta implementação, o RMI apresentou os menores tempos de chamada nas duas operações de busca, seguido pelos Sockets e pelo RPC. Esse resultado deve ser interpretado especificamente dentro das condições do experimento, pois o desempenho também depende da implementação de cada método, da quantidade de registros retornados e da representação utilizada para transmitir os dados.

5.2 Separação entre comunicação e impressão no RMI

A correção da medição também permitiu observar com maior clareza a diferença entre o tempo necessário para realizar uma chamada remota e o tempo gasto apenas para apresentar os dados no terminal.

Operação RMI	Consultas retornadas	Tempo da chamada	Tempo de impressão
Toda a agenda	10000	55,303 ms	1065,163 ms
Busca por CPF	2012	23,237 ms	207,823 ms
Busca por CRM	3346	28,481 ms	374,073 ms

Tabela 2: Separação entre o tempo da chamada RMI e o tempo de impressão dos resultados.

Na consulta de toda a agenda, foram retornados 10.000 registros. A chamada RMI completa levou 55,303 ms, enquanto a impressão dessas consultas no terminal consumiu 1065,163 ms. Isso significa que a exibição dos resultados levou um tempo muito superior ao necessário para realizar a própria chamada remota.

O mesmo comportamento aparece nas buscas específicas. Para CPF, a chamada remota levou 23,237 ms, enquanto a impressão dos 2012 resultados consumiu 207,823 ms. Para CRM, a chamada levou 28,481 ms e a impressão dos 3346 registros levou 374,073 ms.

Também é possível observar que o tempo da chamada aumenta conforme cresce a quantidade de informações retornadas. A busca por CPF retornou 2012 consultas

em 23,237 ms, a busca por CRM retornou 3346 em 28,481 ms e a consulta de toda a agenda retornou 10.000 registros em 55,303 ms. Esse comportamento é compatível com o aumento da quantidade de objetos que precisam ser processados, serializados, transmitidos e desserializados.

Os resultados também demonstram a importância de separar o tempo de comunicação do tempo de entrada e saída no terminal. A impressão de milhares de registros pode consumir significativamente mais tempo do que a própria chamada distribuída e, caso seja incluída na mesma medição, pode levar a uma interpretação incorreta do desempenho da tecnologia.

De forma geral, os testes mostram que nenhuma das três tecnologias deve ser considerada mais rápida em todos os cenários apenas com base em sua arquitetura. Na implementação analisada, o RMI apresentou o menor tempo para os 10.000 agendamentos e para as duas buscas medidas, os Sockets apresentaram resultados intermediários e o XML-RPC apresentou maior tempo nas operações avaliadas. Esses resultados representam o comportamento específico das implementações desenvolvidas e devem ser relacionados às diferenças de comunicação, representação dos dados e processamento existentes em cada arquitetura.

6 Conclusão e Casos de Uso Práticos

A implementação e a medição das três arquiteturas evidenciaram o compromisso entre velocidade, abstração e escalabilidade nos Sistemas Distribuídos. Os resultados provam que a eficiência de rede não depende apenas do peso dos dados trafegados, mas crucialmente da gestão da infraestrutura de conexões.

No contexto de uso prático, a versão em **Sockets** é indicada apenas para cenários de tempo real restrito, como telemetria IoT, onde o controle absoluto do pacote é vital. A versão **RMI** provou ser a melhor escolha para uma arquitetura corporativa fechada, puramente Java, que exija estabilidade na recepção massiva de dados. Por fim, o **RPC** demonstrou ser a arquitetura mais equilibrada para o mercado de desenvolvimento moderno. Apesar do *overhead* de inserção documentado, sua interoperabilidade garante que o núcleo do sistema hospitalar possa ser expandido sem amarrar futuros clientes à linguagem Java.